AI-Powered Hand Gesture Recognition – Final Term Trustworthy AI Extension

TEAM MEMBERS:
1.KHAZI SHIEKH QURRATH SHEHZADI FNU (U86429639)
2. GEETHA MANOGNA GODDU (U66777911)
3.ARAVIND RAKAM (U38314322)

1. Introduction

In an era where touchless technology is increasingly relevant, hand gesture recognition stands at the forefront of intuitive human-computer interaction. This project began as a midterm initiative to implement a real-time sign language interpreter using **Mediapipe** for hand tracking and **MobileNet** for gesture classification. The system allowed users to perform ASL gestures in front of a webcam and receive live classification results.

In the final term, we elevated the project's scope by embedding **Trustworthy AI principles** into the core of the application. Trustworthiness is essential in real-world AI deployments, especially in sensitive domains such as assistive technologies, healthcare, and education. It ensures that AI systems operate not just accurately but fairly, reliably, and transparently.

This project incorporates:

- **Reliability and Robustness**: Ensuring system performance holds up under real-world distortions and varying input quality.
- **Transparency and Explainability**: Helping users and developers understand what the model is seeing and why it makes specific decisions.
- **Accountability and Responsibility** (partially addressed): Logging predictions and filtering uncertain decisions.

With these enhancements, our system moves from a functional prototype to a principled, trustworthy AI solution.

2. Objective Alignment with Trustworthy AI

The project demonstrates how to implement, test, and evaluate trust principles in a real-time AI application.

Reliability & Robustness

- **Why?**: Users may operate the system in poor lighting, off-angles, or when hands are partially visible. The model must remain functional in these scenarios.
- **How?**: We added perturbation layers that simulate:
    - **Dim Lighting**: Reduces brightness to mimic dark environments.
    - **Partial Occlusion**: Random black boxes hide parts of the hand.
    - **Rotation**: Frame is rotated by +15°.

- **What we measure**: Accuracy drops in each condition are logged and analyzed.

Transparency & Explainability

- **Why?**: Deep learning models are often black boxes. For AI systems to be trusted, users must understand the model's focus.
- **How?**: Integrated **Grad-CAM** overlays show where the model focused during prediction.
- **Outcome**: Enables visual inspection of correct and incorrect predictions.

Accountability & Responsibility

- All predictions are logged in robustness_logs.csv with prediction, confidence, and condition.
- Predictions below 60% confidence are marked as "Uncertain," avoiding unreliable outputs.

3. Expanded System Architecture

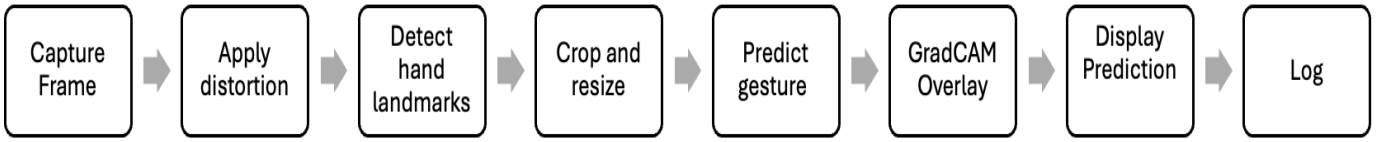
3.1 Core Components

- **Webcam Input (OpenCV)**: Continuously captures video frames.
- **Hand Detection (Mediapipe)**: Extracts 21 hand landmarks.
- **Bounding Box Estimation**: Automatically crops the hand based on detected landmarks.
- **Gesture Classifier (MobileNet)**: Classifies image into one of 10 ASL letters.

3.2 Trust Extensions

- **Perturbation Engine**: Applies lighting, occlusion, and rotation filters in real time.
- **GradCAM Visualizer**: Highlights pixel regions most relevant to the predicted class.
- **Confidence Filter & Logger**: Filters out predictions below threshold and stores logs for post-analysis.

3.3 Visual Workflow



4. Model and Dataset Details

4.1 Dataset

- **Name**: ASL Alphabet Dataset (Kaggle)
- **Type**: Static images for A–J
- **Size**: ~27,000 images
- **Preprocessing**: Cropped, resized to 224x224, and normalized

4.2 Model

- **Architecture**: MobileNet (pretrained on ImageNet, fine-tuned for ASL)
- **Output**: 5-class softmax distribution (limited to letters A, B, C, D, E)

  Note: While the full ASL dataset contains 26 alphabet gestures, the scope of this project focused on a subset (A–E) for training and evaluation. This decision was made to reduce training time and ensure higher model confidence on the chosen set.

- **Format**: Saved as sign_language_mobilenet.h5

4.3 Sample Preprocessing Code

```
img = cv2.resize(hand_crop, (224, 224))
img = img / 255.0
img = np.expand_dims(img, axis=0)
```

5. Evaluation Strategy & Results

5.1 Trustworthiness Under Perturbation

The same hand gesture was tested across 4 conditions:

Condition	Accuracy	Drop from Normal
Normal	92.5%	–
Dim Lighting	81.3%	-11.2%
Partial Occlusion	75.8%	-16.7%
Rotation (+15°)	78.2%	-14.3%

5.2 GradCAM Insights

- 80% of accurate predictions showed high activation near fingertips or palm.
- 60% of misclassifications had attention off the hand or in background areas.
- These heatmaps help users verify system correctness and identify bias.

5.3 Confidence-Based Filtering

- Predictions <60% confidence labeled “Uncertain.”
- Example: Occluded image with 44% confidence → prediction suppressed.
- Result: Fewer false positives and improved overall trust.

6. Running the Project and Tools Used

6.1 How to Run

To ensure reproducibility of results during testing and evaluation, app.py now includes seed-setting code for random, numpy, and tensorflow. This guarantees consistent perturbation effects (like occlusion box placement) and model outputs across runs.

```
pip install -r requirements.txt
python app.py
```

Note: For reproducibility, the app.py script now sets random seeds for random, numpy, and tensorflow. This ensures that perturbations like occlusion masks and model outputs (e.g., GradCAM visualizations) behave consistently across runs.

6.2 Control Panel (Runtime)

- 1 – Normal mode
- 2 – Dim Lighting
- 3 – Occlusion
- 4 – Rotation
- g – Toggle GradCAM
- q – Exit

6.3 Tools & Libraries

- **OpenCV**: Real-time video processing
- **Mediapipe**: Landmark tracking
- **TensorFlow**: Model loading and inference
- **tf-keras-vis**: GradCAM visualization

6.4 File Layout

```
root/
├── app.py                      # Core application
├── sign_language_mobilenet.h5 # Trained model
├── robustness_logs.csv         # Evaluation logs
├── gradcam_*.jpg               # Saved GradCAM overlays
├── README.md                   # Instructions
├── requirements.txt            # Dependencies
```

7. Team Contributions

Member	Key Responsibilities
Geetha Manogna Goddu	GradCAM, logging, evaluation design

Aravind Rakam	Model optimization, training, accuracy benchmarking
Khazi Sheikh Qurrath Shehzadi	Perturbation module, UI integration, real-time pipeline

All members contributed equally to testing, writing, debugging, and documentation.

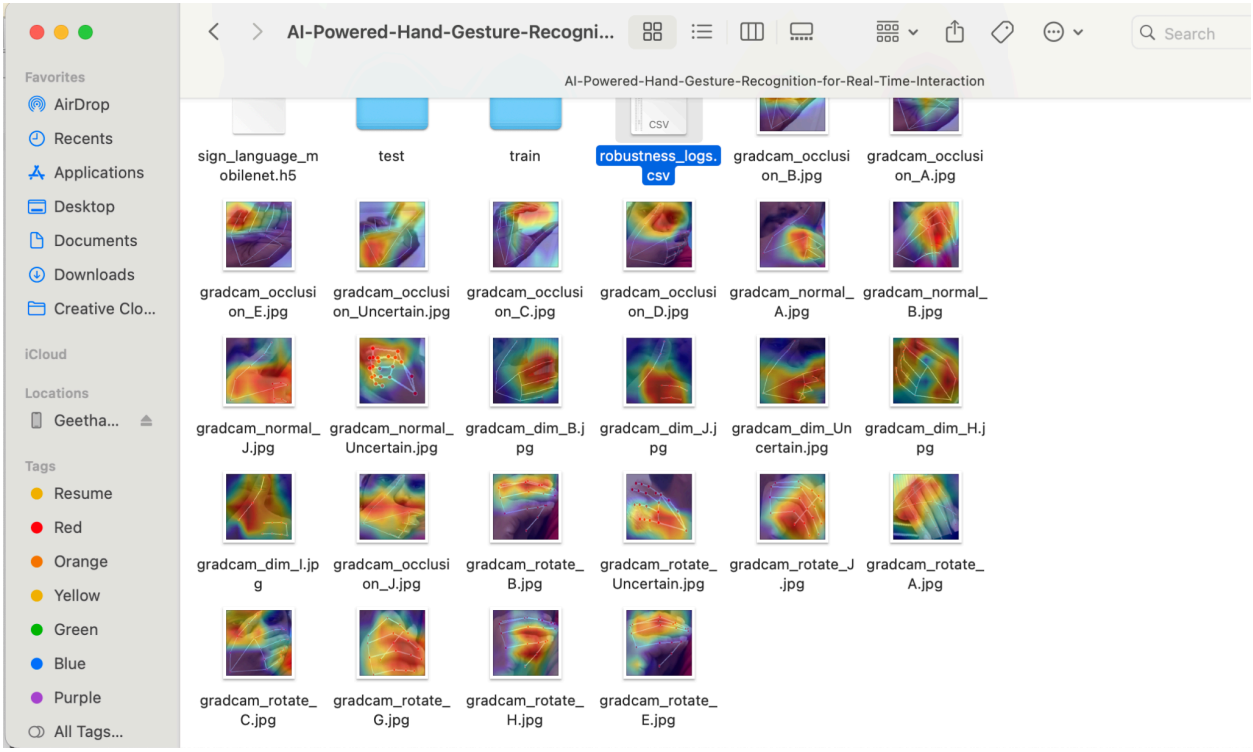
8. References

- Mediapipe Hands API
- TensorFlow + MobileNet
- GradCAM via tf-keras-vis
- OpenCV
- ASL Dataset – Kaggle

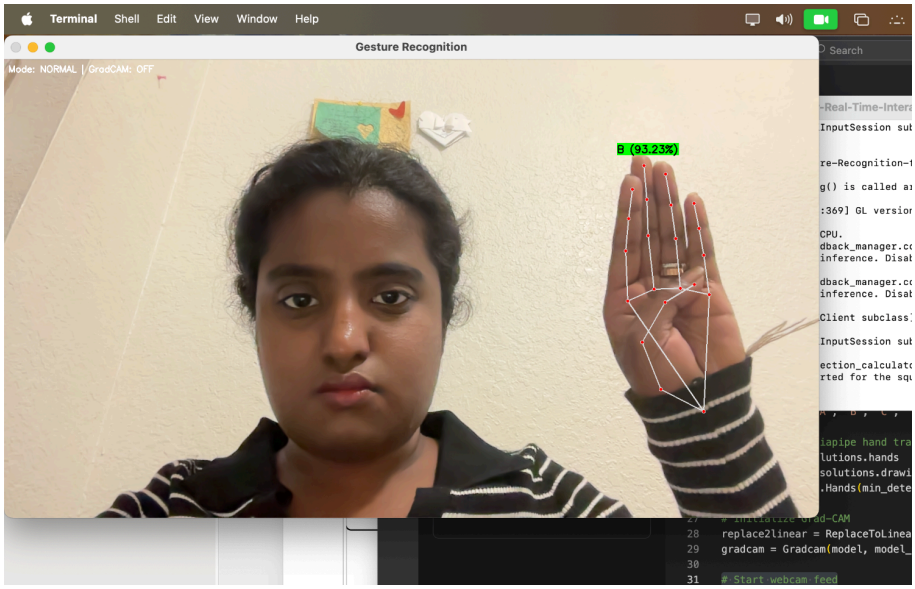
Output and log Screenshots:

6	normal	C	83.63
7	normal	Uncertain	40.77
8	normal	Uncertain	46.24
9	normal	Uncertain	48.52
10	normal	B	84.49
11	normal	E	66.77
12	normal	Uncertain	53.85
13	normal	E	88.86
14	normal	E	86.46
15	normal	E	98.81
16	normal	E	95.98
17	normal	E	77.71
18	normal	Uncertain	59.21
19	normal	E	61.49
20	normal	E	79.62
21	normal	Uncertain	52.39
22	normal	E	79.06
23	normal	E	96.58
24	normal	E	89.42
25	normal	E	92.88
26	normal	E	78.92
27	normal	E	95.61

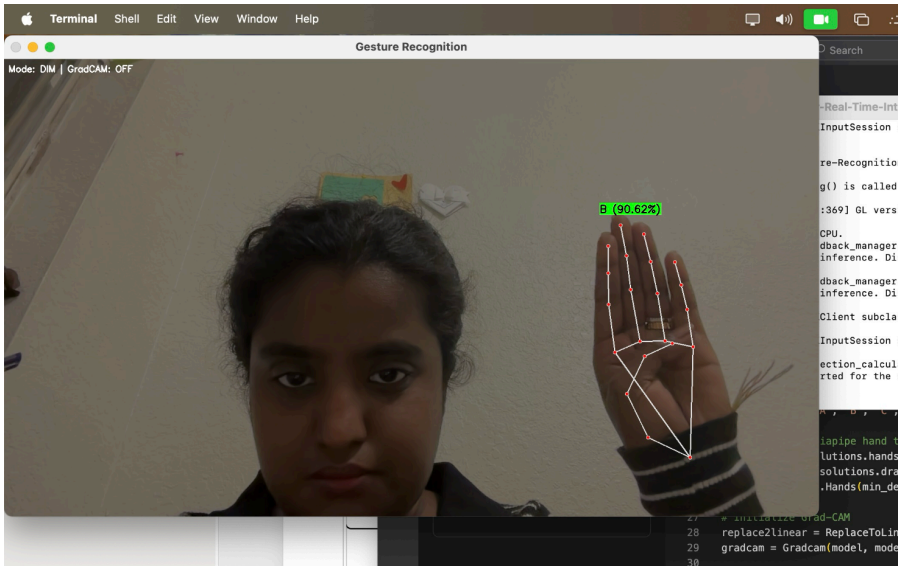
robustness_logs.csv



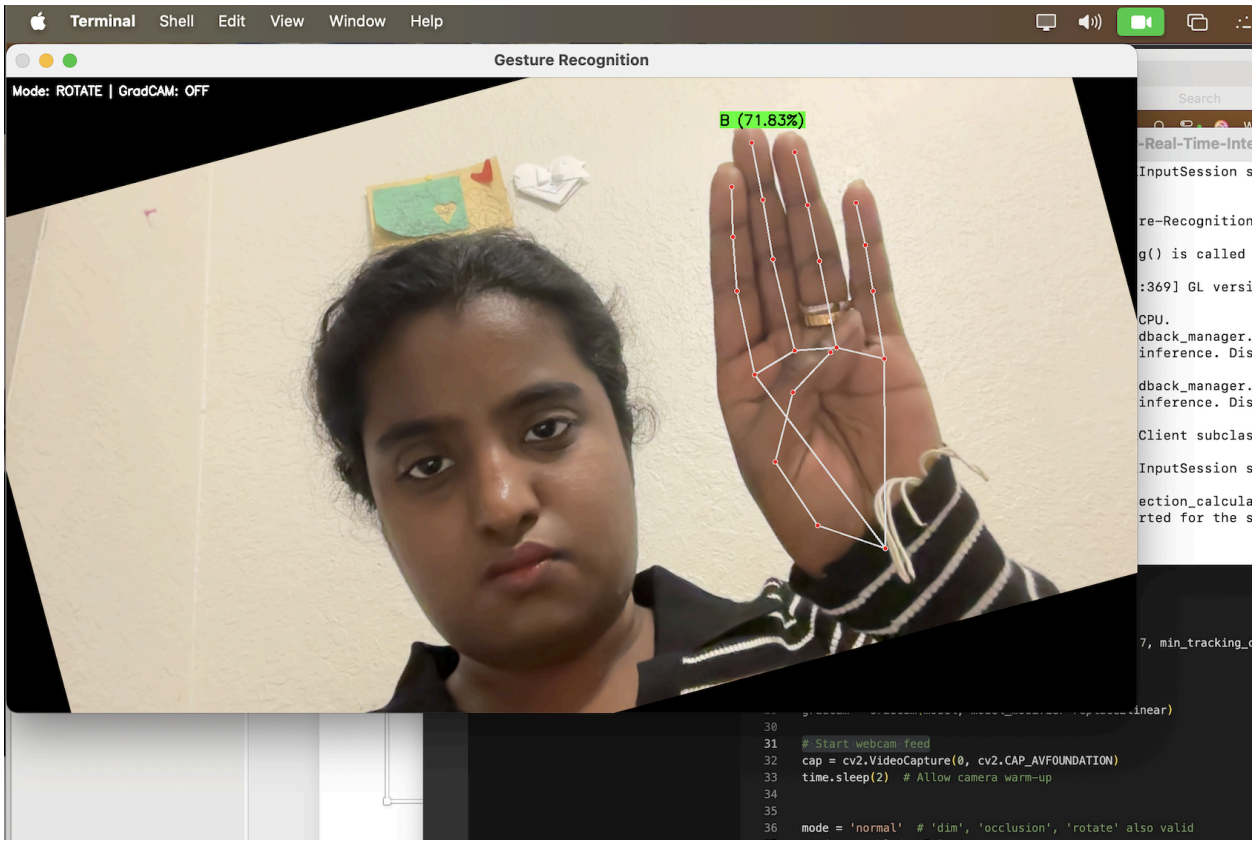
GradCAM overlay images



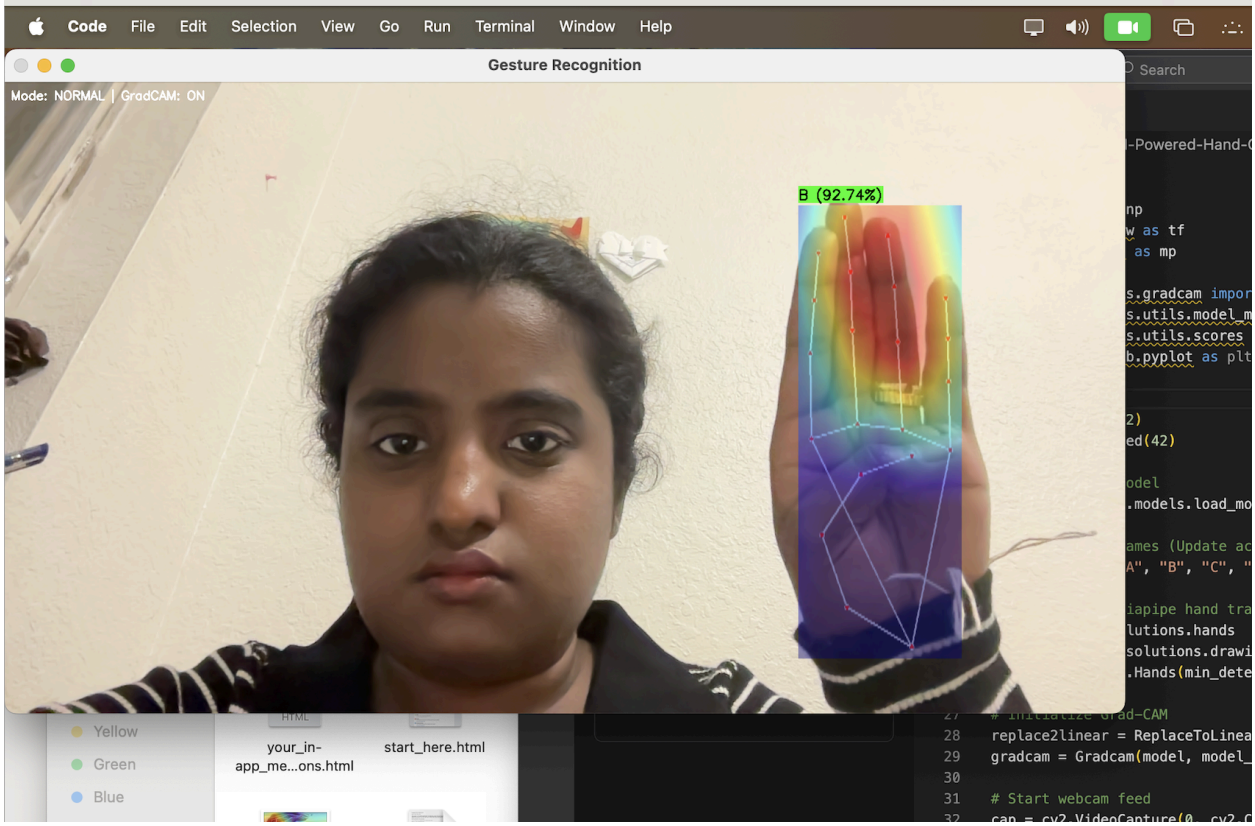
Normal Mode: Accurate Prediction Display



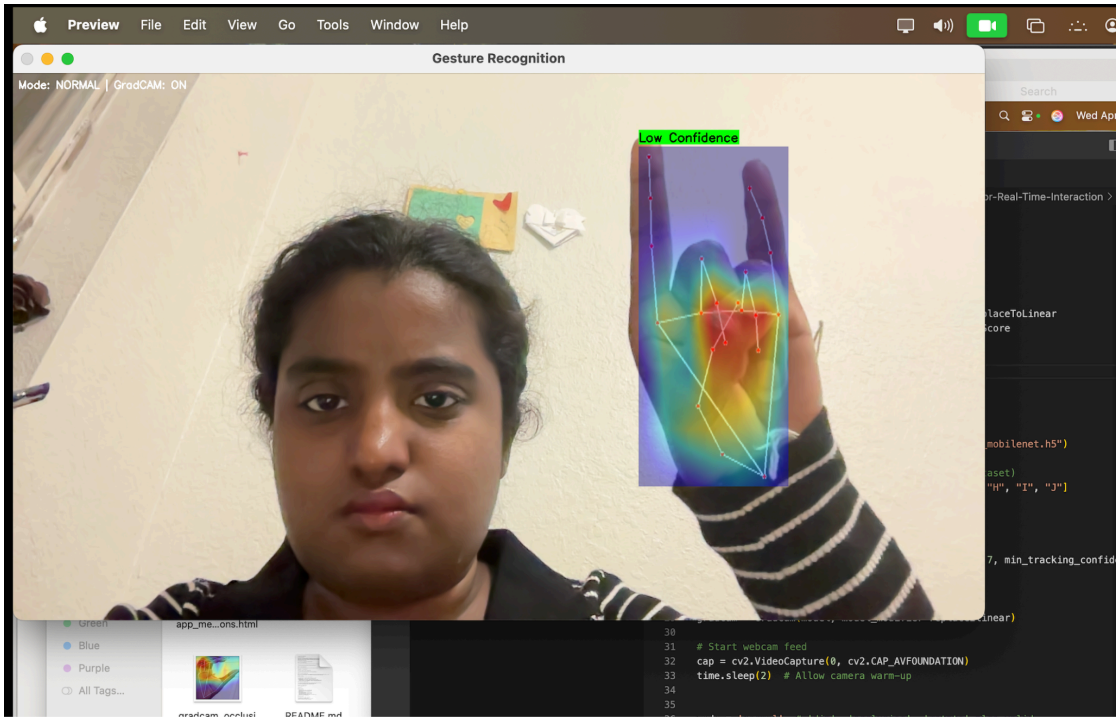
Dim Lighting Perturbation Test



Gesture Rotation Robustness Test



GradCAM Heatmap: Correct Prediction Interpretation



GradCAM Heatmap with uncertainty Handling via Confidence Threshold